



Security Assessment & Formal Verification Final Report



Suilend Liquid Staking

February 2026

Prepared for Suilend

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Threat and Security Overview.....	4
1. Governance & Privileged Access Risks.....	4
2. Exchange-Rate & Accounting Risks.....	4
Findings Summary.....	6
Severity Matrix.....	6
Detailed Findings.....	7
Low Severity Issues.....	8
L-01. Unstaking from Active Stake does not Require Adjusting the Amount to Unstake.....	8
L-02. Rebalancing reverts when one of the configured validator has become inactive.....	9
L-03. Joining Stake may Abort due to Missing Exchange Rate Entry at Current Epoch.....	10
L-04. Lost MIST on Unstaking.....	15
L-05. Fees can Eat Redemptions.....	18
Informational Issues.....	19
I-01. Unreachable branch during validator stake increase.....	19
Formal Verification.....	20
Verification Methodology.....	20
Verification Notations.....	21
General Assumptions and Simplifications.....	21
Formal Verification Properties Overview.....	22
Detailed Properties.....	25
Liquid Staking.....	25
P-L01. System Solvency.....	25
P-L02. Solvency Ratio Monotonicity.....	26
P-L03. No SUI iff no LST.....	26
P-L04. Value Conservation.....	27
P-L05. No Arbitrage Opportunity.....	28
P-L06. Supply Increase Decrease Integrity.....	28
P-L07. Fee Accounting Integrity.....	29
Storage.....	31
P-S01. Total SUI Supply Accounting Correct.....	31
P-S02. Validators Consistency.....	32
P-S03. Validator Management Integrity.....	33
Disclaimer.....	34
About Certora.....	34

Project Summary

Project Scope

Project Name	Repository (link)	First Commit Hash	Latest Commit Hash	Platform
Suilend Liquid Staking	https://github.com/suilend/liquid-staking	36ed8c2	49f75738	Sui

Project Overview

This document describes the specification and verification of **Suilend Liquid Staking** using the Certora Prover. The work was undertaken from **November 11th 2025** to **January 31th 2026**.

All contracts within `contracts/sources` are considered in scope, with an emphasis on the main entry points `storage.move`, `liquid_staking.move`.

The Certora Prover demonstrated that the implementation of the Move contracts above is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Move contracts code, as listed on the following pages.

Protocol Overview

Suilend Liquid Staking provides a framework to facilitate the creation of LSTs on Sui. Anyone has the ability to create their own LST once they have an active validator. Users can then mint LSTs by depositing SUI in the protocol. Doing so, the deposited assets will sit in a pool until the LST administrator effectively pushes them in the validator. At any moment, users have the ability to redeem their LST and immediately unstake their principal as well as potential staking rewards earned.

Threat and Security Overview

This threat model covers the protocol's LST mint/redeem logic (`liquid_staking.move`), stake custody and accounting (`storage.move`), and its dependency on Sui System staking. The analysis groups threats into risk categories, describing how each risk manifests, which components are affected, and which invariants or rules mitigate the threat.

1. Governance & Privileged Access Risks

Affected Components: AdminCap, fee configuration, validator stake management, metadata updates.

Risk Description: The protocol relies on a privileged admin cap to set fees, move stake between validators, collect fees, and update metadata. If the admin key is compromised or misused, the protocol's economic parameters and validator exposure can be arbitrarily changed, leading to user losses or service disruption.

- **Admin Misconfiguration:** Fees (mint/redeem/spread) can be set to punitive values, effectively extracting user value or making the protocol unusable.
- **Validator Stake Control:** `increase_validator_stake` / `decrease_validator_stake` can concentrate stake into risky validators or cause unstake pressure at inopportune times.
- **Fee Collection Timing:** `collect_fees` can be called at any time; if accounting is stale, users may be surprised by effective pricing.

Impact: Protocol-wide risk, since privileged actions alter pricing and validator exposure across all users.

2. Exchange-Rate & Accounting Risks

Affected Components: `storage.move` (`total_sui_supply`, `refresh_validator_info`, `get_sui_amount`), LST conversion functions in `liquid_staking.move`.

Risk Description: The protocol uses integer math and exchange-rate snapshots to value shares and compute SUI/LST conversions. Rounding, dust, and non-distributive floor division can introduce subtle supply drift or fairness issues.

- **Rounding Drift on Partial Active Unstake:** `refresh_validator_info` uses `get_sui_amount` (floor division). During partial active-stake redemptions, floor rounding is not distributive and can reduce `total_sui_supply` over time.
- **Share/Asset Conversion Rounding:** `sui_amount_to_lst_amount` and `lst_amount_to_sui_amount` are integer divisions; sequential redemptions can favor later redeemers by capturing residual dust.
- **Dust Handling:** If `exchange_rate.sui_amount == 0` or `pool_token_amount == 0`, `get_sui_amount` returns 1:1, which can misprice shares when dust exists.

Impact: Slow value leakage, skewed redemption outcomes, and potential under-collateralization in edge cases.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	5	5	4
Informational	1	1	0
Total	6	6	4

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
L-01	Unstaking from Active Stake does not Require Adjusting the Amount to Unstake	Low	Fixed
L-02	Rebalancing Reverts when one of the Configured Validators has Become Inactive	Low	Acknowledged
L-03	Joining Stake may Abort due to Missing Exchange Rate Entry at Current Epoch	Low	Fixed
L-04	Lost MIST on Unstaking	Low	Fixed
L-05	Fees can Eat Redemptions	Low	Fixed
I-01	Unreachable Branch during Validator Stake Increase	Informational	Acknowledged

Low Severity Issues

L-01. Unstaking from Active Stake does not Require Adjusting the Amount to Unstake

Severity: **Low**

Impact: **Medium**

Likelihood: **Low**

Files:
[storage.move#L376](#)

Status: Fixed

Description: When `unstake_approx_n_sui_from_active_stake` is executed with a `target_unstake_sui_amount` less than `MIN_STAKE_THRESHOLD` (`1e9`) the logic will replace the SUI amount to unstake to `MIN_STAKE_THRESHOLD` even though the initial amount can be unstaked. Operations relying on this invocation could produce minor side effects on the protocol, such as unstaking 1 SUI from the validator instead of dust. These operations include the redemption of LST (`liquid_staking::redeem`) as well as the decrease of a validator stake (`liquid_staking::decrease_validator_stake`). Since these functions unstake using a `FungibleStakedSui`, there is no restriction in the amount of SUI unstaked, contrary to `StakedSui`.

Scenario: Let the following state:

- `unstake_approx_n_sui_from_active_stake` is executed with
`target_unstake_sui_amount == 1000`
- `total_sui_amount == 1e9`

Here, withdrawing 1000 mist from the active validator should succeed and leave (`1e9 - 1000`) on the validator but the `target_unstake_sui_amount` amount is increased to `MIN_STAKE_THRESHOLD`. Then, `if (total_sui_amount <= target_unstake_sui_amount)` is evaluated to `true` and the underlying branch is entered which will effectively withdraw all `1e9` SUI from the validator instead of only 1000.

Recommendation: Remove the branch responsible for adjusting the amount to unstake from the active stake since it is not necessary as it deals with `FungibleStakedSui`.

Customer's response: Fixed in commit [d8684eaf](#).

Fix Review: Fix confirmed.

L-02. Rebalancing reverts when one of the configured validator has become inactive

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:
[weight.move#L232](#)

Status: Acknowledged

Description: The `weight.move` module maintains a `VecMap` of `validator_addresses_and_weights` configured by the owner of a `WeightHookAdminCap`. This `VecMap` stores the cut each configured validator should have. By the end of the `rebalance_internal` function, a call to `liquid_staking_info.increase_validator_stake` is made to actually stake the corresponding amount to every validator in `validator_addresses_and_weights`. However, this call may attempt to stake `SUI` to an inactive validator which will result in an abort. A validator can become inactive when a user redeems enough `LST` to push its stake under the minimum threshold. The issue stems from the fact that `validator_addresses_and_weights` fails to take into consideration the validators state and assumes validators will remain active. In fact, this will disrupt the rebalancing up until the inactive validator is removed from `validator_addresses_and_weights` or the validator becomes active again.

Recommendation: Either:

- implement a filter responsible for automatically setting the weight of a validator that is not part of the `LST` anymore but still in `validator_addresses_and_weights` to 0.
- document that this is the responsibility of the `WeightHookAdminCap` to update the validator addresses and weights using `set_validator_addresses_and_weights` when one of the validators has become inactive.

Customer's response: Acknowledged.

L-03. Joining Stake may Abort due to Missing Exchange Rate Entry at Current Epoch

Severity: Low	Impact: Low	Likelihood: Low
Files: storage.move#L578	Status: Fixed	

Description: The `get_or_add_validator_index_by_staking_pool_id_mut` function (and all functions relying on it) is subject to failure due to an attempt at accessing an entry (using the `borrow` API) that doesn't exist in a `Table`. The code assumes there is always an exchange rate entry at every epoch in a staking pool, starting at its activation epoch. However, attempting to `borrow()` a staking pool's exchange rate at the current epoch can commonly fail in the very specific case where the staking pool just got activated.

In a more extreme case, Sui can enter "safe mode" during epoch rollover and skip exchange rate updates (see [safe mode test](#)).

The Sui framework is implemented in such a way that there is always a gap in a new staking pool's exchange rate, right after its activation epoch. In the Sui framework code, `exchange_rates` snapshots are written at epoch boundaries, through `validator_set::advance_epoch()`.

Rust

```
// validator_set.move public(package)
fun advance_epoch(...) {
    process_pending_stakes_and_withdraws(&mut self.active_validators, ctx);
    // --- snip ---
    self.update_validator_positions_and_calculate_total_stake(...);
}
```

This function first processes currently active validators by invoking `process_pending_stakes_and_withdraws()`, setting their next epoch's `PoolTokenExchangeRate`.

Rust

```
// staking_pool.move public(package)
fun process_pending_stakes_and_withdraws(pool: &mut StakingPool, ctx: &TxContext) {
    let new_epoch = ctx.epoch() + 1;
    // --- snip ---
    pool.exchange_rates.add(new_epoch, PoolTokenExchangeRate {
        sui_amount: pool.sui_balance, pool_token_amount: pool.pool_token_balance,
    });
}
```

Then, it activates pending active validators by invoking `self.update_validator_positions_and_calculate_total_stake()` which triggers `staking_pool::activate_staking_pool()` under the hood, setting their exchange rate at `ctx.epoch()` (the current epoch that is ending).

Rust

```
// validator_set.move
fun update_validator_positions_and_calculate_total_stake(...) {
    // --- snip ---
    pending_active_validators.do!(|mut validator| {
        let validator_stake = validator.total_stake();
        let voting_power = voting_power::derive_raw_voting_power(
            validator_stake,
            initial_total_stake,
        );
        if (voting_power >= min_joining_voting_power_threshold) {
            @> validator.activate(ctx.epoch());
            event::emit(ValidatorJoinEvent {
                epoch: ctx.epoch(),
                validator_address: validator.sui_address(),
                staking_pool_id: validator.staking_pool_id(),
            });
            self.active_validators.push_back(validator);
        }
        // --- snip ---
    })

    // staking_pool.move
    public(package) fun activate_staking_pool(pool: &mut StakingPool, activation_epoch: u64) {
        // Add the initial exchange rate to the table.
        pool.exchange_rates.add(activation_epoch, initial_exchange_rate());
        // --- snip ---
        pool.activation_epoch.fill(activation_epoch);
    }
}
```

Scenario: Assume:

- Epoch rollover is happening from $E=4$ to $E+1=5$
- A validator V becomes active for the first time in epoch 5

During rollover (still `ctx.epoch() == 4`):

- Existing active validators write `exchange_rates[5]` (because `new_epoch = 4 + 1`)
- V is activated and its pool writes **only** `exchange_rates[4]` via `activate_staking_pool(activation_epoch=4)`

Now epoch 5 begins (`ctx.epoch() == 5`).

If `get_or_add_validator_index_by_staking_pool_id_mut` is called for V during epoch 5, it executes:

```
Rust
let exchange_rates = system_state.pool_exchange_rates(&staking_pool_id);
let latest_exchange_rate = exchange_rates.borrow(ctx.epoch()); // borrow(5)
```

But V .`exchange_rates` only contains the activation entry at key 4, not key 5, so `borrow(5)` will abort.

Impact: The execution of `liquid_staking::increase_validator_stake` may be disrupted until epoch rollover for validators that got activated and have an LST created at that same epoch.

Proof of Concept:

```
Rust
#[test]
fun test_failure_due_to_exchange_rate_gap() {
    let mut scenario = test_scenario::begin(@0x0);

    // this will make the validator active
    setup_sui_system(&mut scenario, vector[100]);

    scenario.next_tx(@0x0);
    let mut system_state = scenario.take_shared<SuiSystemState>();
    sui_system::set_epoch_for_testing(&mut system_state, scenario.ctx().epoch() + 1);
```

```
scenario.next_epoch(@0x0);

let pool_id = system_state.validator_staking_pool_id(@0x0);
let rates = system_state.pool_exchange_rates(&pool_id);
print_exchange_rates(rates, scenario.ctx().epoch());

let (admin_cap, mut lst_info) = create_lst<TEST>(
    fees::new_builder(scenario.ctx())
        .set_sui_mint_fee_bps(100)
        .set_redeem_fee_bps(100)
        .to_fee_config(),
    coin::create_treasury_cap_for_testing(scenario.ctx()),
    scenario.ctx()
);

assert!(lst_info.total_lst_supply() == 0, 0);
assert!(lst_info.storage().total_sui_supply() == 0, 0);

let sui = coin::mint_for_testing<SUI>(200 * MIST_PER_SUI, scenario.ctx());
let lst = lst_info.mint(&mut system_state, sui, scenario.ctx());

debug::print(&string::utf8(b"about to call increase_validator_stake (expected abort)"));
lst_info.increase_validator_stake(
    &admin_cap,
    &mut system_state,
    @0x0,
    200 * MIST_PER_SUI,
    scenario.ctx()
);

test_scenario::return_shared(system_state);
sui::test_utils::destroy(admin_cap);
sui::test_utils::destroy(lst);
sui::test_utils::destroy(lst_info);

scenario.end();
}
```

Recommendation: Avoid executing `exchange_rates.borrow(ctx.epoch())` and instead, reuse `storage::get_latest_exchange_rate` to search the latest exchange rate snapshotted by Sui.



Customer's response: Fixed in commit [16e21597](#).

Fix Review: Fix confirmed.

L-04. Lost MIST on Unstaking

Severity: Low	Impact: Low	Likelihood: Medium
Files: storage.move#L492-L500 storage.move#L219-L227	Status: Fixed	Violated Properties: P-L01 , P-L02 , P-L03 , P-L04 , P-L06 , P-L07

Description: The `total_sui_supply` accounting can decrease during unstaking operations due to a truncating division in exchange rate conversions. This can occur in the functions that trigger unstaking, which include `decrease_validator_stake`, `collect_fees`, and `redeem`.

The protocol tracks `total_sui_supply` as the sum of the internal SUI pool balance plus the SUI-equivalent value of all staked positions across validators (active and inactive). For active stake, the SUI-equivalent value is computed by converting fungible staked-SUI `token_amount` using the validator exchange rate via `get_sui_amount`, which performs truncating division (rounding down):

```
Rust
let res = (exchange_rate.sui_amount() as u128)
    * (token_amount as u128)
    / (exchange_rate.pool_token_amount() as u128);
```

The issue occurs in `split_from_active_stake`, which in summary does the following

1. Split `k` tokens from a validator's active stake of `m`, leaving `m-k`
2. Call `refresh_validator_info`, which recomputes the validator's active-stake SUI value as `get_sui_amount(exchange_rate, m - k)`
3. Redeems the `k` tokens for `k'` SUI, where `k' <= get_sui_amount(exchange_rate, k)`, and adds it to the pool.

As a result, after the split, the protocol tracks the validator's remaining stake value as `get_sui_amount(m - k)` while the pool's balance increases by `get_sui_amount(k)`. Since `get_sui_amount` uses the truncating division, it is not distributive, i.e., we have

Rust

```
get_sui_amount(rate, k) + get_sui_amount(rate, m-k) <= get_sui_amount(rate, m)
```

This can cause the aggregate `total_sui_supply` (pool + stakes) to decrease by the rounding error.

Since the rounding error is at most one, each unstaking operation can “lose” at most 1 MIST. However, the process can be repeated arbitrarily many times, so the loss accumulates linearly with the number of unstake operations (≤ 1 MIST per unstaking per validator).

This affects the backing of LSTs: because `total_sui_supply` is used in the LST exchange-rate calculation, every reduction in `total_sui_supply` makes each LST worth slightly less SUI than it should be. The depreciation is negligible, but in the worst case, when the system is near a 1:1 SUI to LST rate, a reduction of the SUI supply can potentially leave the protocol insolvent. That is, in a state where there are more LSTs outstanding than SUI balance to redeem them.

Recommendation: Since the typically only dust amounts of SUI are affected, add a check that aborts execution whenever unstaking would push the protocol into an insolvent state.

Customer’s response: Acknowledged and fixed in commit [0a8c7449](#) and [b5df3d24](#).

Fix Review: Fixed by redeeming the `FungibleStakedSui` first and then updating internal accounting using the actual `redeemed_amount` returned by `system_state.redeem_fungible_staked_sui(..)`.

When unstaking active stake, using `split_from_active_stake` or `take_active_stake`, the system now calls the function `redeem_and_update_accounting` instead of refreshing validator info before redeeming:

Rust

```
fun redeem_and_update_accounting(
    self: &mut Storage,
    system_state: &mut SuiSystemState,
    validator_index: u64,
    fungible_staked_sui: FungibleStakedSui,
    ctx: &TxContext
): Balance<SUI> {
    let redeemed_sui = system_state.redeem_fungible_staked_sui(fungible_staked_sui, ctx);
```

```
let redeemed_amount = redeemed_sui.value();

// Update accounting based on actual redeemed amount.
// The validator's total_sui_amount may drift slightly from get_sui_amount()
// but this will be reconciled during the next epoch refresh
let validator_info = &mut self.validator_infos[validator_index];
self.total_sui_supply = self.total_sui_supply - redeemed_amount;
validator_info.total_sui_amount = validator_info.total_sui_amount - redeemed_amount;
redeemed_sui
}
```

Since floor arithmetic isn't additive, this can leave small "dust" counted in `total_sui_supply` after unstaking, temporarily overstating backing until the next epoch refresh reconciles it. This is acceptable because it is bounded and temporary, but it means that the following accounting invariants only hold across epoch boundaries rather than at every intermediate step within an epoch:

- Validators recorded `total_sui_supply` precisely matches the sum of (active and inactive) staked SUI ([P-S01](#))
- Validators with no active or inactive stake have zero total SUI recorded ([P-S02](#))

Apart from that, all rules previously violated by this finding ([P-L01](#), [P-L02](#), [P-L03](#), [P-L04](#), [P-L06](#), [P-L07](#)) are passing.

L-05. Fees can Eat Redemptions

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:
[liquid_staking.move](#)
[#L298-L304](#)

Status: Fixed

Violated Property: [P-L07](#)

Description: When redeeming LSTs for SUI, a fee is deducted from the SUI payout amount:

```
Rust
let redeem_fee_amount = if (is_custom_redeem) {
    self.fee_config.get().calculate_custom_redeem_fee(sui.value())
} else {
    self.fee_config.get().calculate_redeem_fee(sui.value())
};
self.fees.join(sui.split(redeem_fee_amount as u64));
```

Because the fee rate is capped at 100% and the fee computation in both paths rounds in the protocol's favor (i.e., ceils the fee amount), it is possible for the post-fee remainder to round down to zero SUI after the split. In that scenario, the redemption burns the user's LST while paying out 0 SUI, effectively transferring the user's entire claim to the protocol via fees.

Recommendation: Abort the redemption if the SUI payout is 0 after fees. Optionally, let the user pass a `min_payout` and abort if the payout is below it.

Customer's response: Acknowledged and fixed in commit [15c9f0d5](#).

Fix Review: Fixed by aborting redemption when the SUI payout amount becomes zero after fees are applied. The corresponding rule [P-L07](#) is passing.

Informational Issues

I-01. Unreachable branch during validator stake increase

Description: When `liquid_staking::increase_validator_stake` is called, the `storage::join_stake` function is invoked to join the stake (`StakedSui`) to the `validator_info`. The logic differentiates 2 cases to join the stake in the correct field: when the stake's activation epoch is a previous epoch or the current epoch (joins to `active_stake`); when the stake's activation epoch is in the future (joins to `inactive_stake`). The problem is that `increase_validator_stake` directly stakes SUI in the validator using `request_add_stake_non_entry` which returns a `StakedSui` with an activation epoch set to the next epoch, in the future. Thus, the stake will systematically be joined to the `inactive_stake`.

Recommendation: Remove the `if` statement from `join_stake` to directly join to `inactive_stake` and document the behavior. The function should be as straightforward as `join_fungible_stake`.

Customer's response: Acknowledged.

Formal Verification

Verification Methodology

We performed verification of the **Suilend** protocol using the Certora Prover which is based on Satisfiability Modulo Theories (SMT).

In short, the Certora Prover works by compiling formal specifications written in the Certora Verification Language (CVLM) and the implementation source code written in Move.

More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVLM holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors/initialization functionality as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.

Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective initialization function. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant. Each such case results in one rule.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

- The protocol relies on primitives provided by the native SUI staking mechanisms. To make verification feasible, we over-approximate certain aspects of this, as follows:
 - We summarize the system function `pool_token_exchange_rate_at_epoch` to return a nondeterministic exchange rate per validator per epoch. We assume that the returned rate is always positive, in the sense that the SUI amount is equal to or exceeds the pool token amount. In other words, we assume that staked SUI is worth at least the same amount in SUI.
 - For each staking pool, we use ghost states to track its total SUI amount as the sum of its principal amount and reward amount. We therefore always assume that an exchange rate's total SUI amount equals the corresponding pool's SUI amount (at a given epoch). To this end, we summarized `convert_to_fungible_staked_sui` to update these ghost states accordingly, i.e., by adding to the newly staked SUI to principal staked amount.
 - We summarize the system function `redeem_fungible_staked_sui` to simply convert the given amount of fungible staked SUI to SUI using an exchange rate satisfying the above conditions.
- Liquid staking pools are parametrized with the type of the liquid staking token they manage. Due to a limitation of the Move prover, we cannot prove inductive invariants for objects with arbitrary type parameters. To work around this limitation, we create an empty type `DummyToken` and use fixed type instantiations `LiquidStakingInfo<DummyToken>`,

for which we create wrapper functions that instantiate the generic types with `DummyToken` and delegate all calls to the actual implementation in the liquid staking module. This approach allows the prover to reason about these types concretely while maintaining the same behavior as the generic implementation.

- By default, we assume that Sui is staked to at most one validator and that all loops are unrolled at most once. For properties that become trivial under this assumption, we use a higher loop unrolling limit. If applicable, we document this in respective property descriptions below.

Formal Verification Properties Overview

ID	Title	Impact	Status
P-LO1	System Solvency	If solvency is not maintained, the protocol can end up with more LST issued than SUI backing it. This can drop the exchange rate below 1 and cause users to receive less SUI than expected (or be unable to redeem fully), with losses effectively spread across LST holders and potentially worsening over time.	Verified after Fix
P-LO2	Solvency Ratio Monotonicity	If the exchange rate can decrease after normal operations, LST holders can be diluted over time and see their tokens become worth less SUI than before.	Verified after Fix
P-LO3	No SUI iff No LST	If either direction of this invariant can be broken, the protocol can end up with orphaned SUI when no LST exists, or with outstanding LST when no SUI backing exists. Both cases indicate accounting inconsistency that can lead to trapped funds or unredeemable tokens.	Verified after Fix

P-L04	Value Conservation	If value is not conserved, protocol operations can create or destroy value through accounting mismatches. This can allow SUI backing to decrease without a corresponding user payout or recorded fee, or cause redemptions to burn value by returning less SUI than the LST redeemed implies.	Verified after Fix
P-L05	No Arbitrage Opportunity	If an immediate mint-then-redeem round trip can return more SUI than deposited, users can extract risk-free profit directly from protocol reserves. This potentially drains backing and pushes losses on remaining LST holders through a worsening exchange rate.	Verified
P-L06	Supply Increase Decrease Integrity	If minting and redemption are not the only ways supplies can change, the protocol can unexpectedly create or destroy value. In particular, if non-redemption functions can reduce the SUI reserve, SUI can leak from the pool without users burning LST.	Verified after Fix
P-L07	Fee Accounting Integrity	If fee accounting constraints are not maintained, the protocol can record spread fees that exceed the SUI reserve, making internal accounting inconsistent. Additionally, if fees are not constrained,, a user can deposit SUI or burn LST yet receive zero value in return, turning fees into a total loss mechanism.	Verified after Fix
P-S01	Total SUI Supply Accounting Correct	If the protocol's tracked total SUI supply can diverge from the SUI actually held in the pool and stakes, key calculations that depend on it become invalid. This can lead to mispriced minting/redemptions and user	Verified

		losses due to incorrect internal accounting.	
P-S02	Validators Consistency	If validator registry invariants are not preserved, the protocol can mis-account stake by double-counting validators, tracking more validators than supported, or leaving non-zero SUI recorded for validators with no stake after refresh. This can lead to invalid total supply and exchange-rate calculations.	Verified
P-S03	Validator Management Integrity	If validator registry operations are not tightly controlled, validators could be added or removed through unexpected code paths. If refreshing logic does not work as expected, stale entries can persist leading to incorrect accounting.	Verified

Detailed Properties

Liquid Staking

Module General Assumptions

Invariant/parametric properties are established by considering all public non-view functions of the liquid staking module.

Module Properties

P-L01. System Solvency

Status: Verified after Fix

Verifies that the liquid staking protocol maintains solvency throughout its lifecycle, ensuring that the SUI backing always meets or exceeds the LST supply (maintaining an exchange rate ≥ 1).

Rule Name	Status	Description	Link to rule report
solvency_base	Verified	<i>Base case I: Verifies that newly created empty liquid staking pools are solvent.</i>	Report
solvency_base_staker	Verified	<i>Verifies that newly created liquid staking pools initialized with existing stake are solvent.</i>	
solvency_step	Verified after Fix	<i>Inductive step: Verifies that all state-modifying operations preserve protocol solvency. Assumes the protocol is solvent in the pre-state and proves it remains solvent after the operation.</i>	

P-LO2. Solvency Ratio Monotonicity

Status: Verified after Fix

This property verifies that the SUI/LST exchange rate is non-decreasing across operations, protecting users from value dilution. The monotonically increasing exchange rate ensures that LST tokens never lose value relative to SUI over time.

Rule Name	Status	Description	Link to rule report
monotonicity	Verified after Fix	<i>Verifies that user operations cannot decrease the SUI/LST rate. In other words, after any operation, the exchange is equal or better than before.</i>	Report

P-LO3. No SUI iff no LST

Status: Verified after Fix

Ensures that an empty LST supply implies an empty SUI reserve (preventing orphaned SUI), and conversely, that an empty SUI reserve implies no outstanding LST tokens (preventing unbacked tokens).

Rule Name	Status	Description	Link to rule report
no_lst_no_sui_base	Verified	<i>Base case: Verifies that newly created liquid staking pools (both empty and with initial stake) satisfy zero LST supply implies zero SUI supply.</i>	Report
no_lst_no_sui_step	Verified	<i>Inductive step: Verifies that all state-modifying operations preserve the invariant that zero LST supply implies zero SUI supply. Assumes the system is solvent and has correct accounting in the pre-state.</i>	
no_sui_no_lst_base	Verified	<i>Base case: Verifies that newly created liquid staking pools (both empty and with initial stake)</i>	

		satisfy that zero SUI supply implies zero LST supply.	
no_sui_no_lst_step	Verified after Fix	Inductive step: Verifies that all state-modifying operations preserve the invariant that zero SUI supply implies zero LST supply. Assumes the system is solvent and has correct accounting in the pre-state.	

P-L04. Value Conservation

Status: Verified after Fix	Ensures that the liquid staking protocol conserves value across all operations. For operations that do not involve deposits or redemptions, the total SUI and LST supplies must be non-decreasing. For mint operations, the deposited SUI value must equal the sum of the LST backing increase and protocol fees collected. For redeem operations, the burned LST value must equal the sum of SUI returned to users and protocol fees.
----------------------------	--

Rule Name	Status	Description	Link to rule report
sui_value_conservation	Verified after Fix	Verifies that for operations that cannot decrease SUI supply (non-redemption operations), the total SUI backing remains non-decreasing. This prevents unauthorized SUI withdrawals.	Report
lst_value_conservation	Verified	Verifies that for operations that cannot decrease LST supply (non-redemption operations), the total LST supply remains non-decreasing. This prevents unauthorized token burns.	
deposit_value_conservation	Verified	Verifies that during mint operations, the deposited SUI value is fully accounted for as either an increase in the SUI backing or as protocol fees. This ensures no value is lost during deposits.	

redeem_value_conservation	Verified after Fix	Verifies that during redemption operations, the decrease in SUI backing equals the sum of SUI returned to the user and protocol fees collected. This ensures no value is lost during redemptions.	
---------------------------	--------------------	---	--

P-L05. No Arbitrage Opportunity

Status: Verified	This property verifies that there are no arbitrage opportunities where a user could mint LST and immediately redeem it for more SUI than initially deposited.
------------------	---

Rule Name	Status	Description	Link to rule report
no_arbitrage_opportunity	Verified	Verifies that no arbitrage opportunity exists.	Report

P-L06. Supply Increase Decrease Integrity

Status: Verified after Fix	Verifies that only authorized operations can increase or decrease the SUI and LST supplies. Specifically, only mint operations can increase supplies, and only redemption operations can decrease supplies. This prevents unexpected minting or burning of tokens, ensuring that supply changes only occur through the intended user-facing deposit and withdrawal flows.
----------------------------	---

Rule Name	Status	Description	Link to rule report
only_redemption_decreases_sui_supply	Verified after Fix	Verifies that only authorized redemption operations can decrease the total SUI supply. This prevents unauthorized withdrawal of SUI backing through non-redemption functions.	Report

only_redemption_decreases_lst_supply	Verified	Verifies that depositing a non-zero amount of SUI always results in receiving non-zero total value (LST tokens plus fees), preventing complete loss of user funds during minting.	
only_minting_increases_sui_supply	Verified	Verifies that only authorized mint operations can increase the total SUI supply backing. This prevents unauthorized increase of SUI backing through non-mint functions.	Report I , Report II
only_minting_increases_lst_supply	Verified	Verifies that only authorized mint operations can increase the total LST supply. This prevents unauthorized creation of LST tokens through non-mint functions.	

P-L07. Fee Accounting Integrity

Status: Verified after Fix	Verifies that the protocol's fee accounting maintains critical invariants: Accrued spread fees must never exceed the total SUI supply (preventing over-collection), fees must grow monotonically except during collection operations (ensuring they are not lost), and fees must never completely consume user deposits or redemptions (guaranteeing users always receive value). These properties ensure the fee mechanism operates correctly without compromising user funds or protocol accounting.
----------------------------	--

Rule Name	Status	Description	Link to rule report
spread_fees_dont_exceed_sui_supply_base	Verified	Base case: Verifies that for newly created pools, accrued spread fees do not exceed the total SUI supply. This is trivial because fresh pools do not have any accrued fees.	Report
spread_fees_dont_exceed_sui_supply_step	Verified after Fix	Inductive step: Verifies that all operations preserve the invariant that accrued spread fees do not exceed the total SUI supply.	

fees_grow_monotonically	Verified	<i>Verifies that accrued fees either remain constant or increase across all operations, except during explicit fee collection. This ensures fees are not lost or incorrectly reduced during operations.</i>	
fees_dont_eat_redemption	Verified after Fix	<i>Verifies that redemption fees do not completely consume the user's redemption, ensuring users always receive a non-zero amount of SUI when redeeming non-zero LST tokens.</i>	
fees_dont_eat_deposit	Verified	<i>Verifies that deposit fees do not completely consume the user's deposit, ensuring users always receive a non-zero amount of LST tokens when depositing non-zero SUI.</i>	

Storage

Module Properties

Invariant/parametric properties are established by considering all public non-view functions of the storage module.

Module Properties

P-S01. Total SUI Supply Accounting Correct

Status: Verified

Verifies that the total SUI supply tracked by the storage in the field `total_sui_supply` precisely matches the sum of all SUI held across different locations: the liquidity pool, active and inactive stake stake across all validators. This property ensures that the protocol's internal accounting accurately reflects the actual SUI holdings.

Note: This invariant only holds across epoch boundaries after refresh, as accounting can temporarily drift within an epoch (refer to the fix review of [L-01](#) for more details).

Rule Name	Status	Description	Link to rule report
total_sui_supply_correct_base	Verified	<i>Base case: Verifies that newly created storage has correct total SUI supply accounting.</i>	Report
total_sui_supply_correct_step	Verified	<i>Inductive step: Verifies that all storage operations preserve the invariant that the stored total SUI supply matches the actual sum across all locations.</i>	

P-S02. Validators Consistency

Status: Verified

Ensures the internal validator registry maintains structural invariants critical for correct protocol operation. Validates that: (1) no duplicate validators exist (by staking pool ID or validator address); (2) the registry size never exceeds the maximum validators limit; (3) after a full refresh, validators with no active or inactive stake have zero total SUI recorded.

Note that to make verification feasible, we use an upper bound of five validators. Additionally, for the verification of boundedness of the validator list, we allow all loops to be unrolled at most six times. For all other rules included in this property, we allow all loops to be unrolled at most twice instead.

Rule Name	Status	Description	Link to rule report
no_duplicate_validators	Verified	<i>Verifies that adding a validator to the registry only occurs if neither its staking pool ID nor its validator address already exists, preventing duplicate entries and double-counting.</i>	Report
no_stake_no_sui_base	Verified	<i>Base case: Verifies that for newly created storage, validators with no active or inactive stake have zero total SUI amount recorded.</i>	
no_stake_no_sui_step	Verified	<i>Inductive step: Verifies that all operations preserve the invariant that validators with no active or inactive stake have zero total SUI recorded.</i> <i>Note: This invariant only holds across epoch boundaries after refresh, as accounting can temporarily drift within an epoch (refer to the fix review of L-01 for more details).</i>	
validators_upper_bound_base	Verified	<i>Base case: Verifies that newly created storage has a validator count within the maximum limit.</i>	Report

validators_upper_bound_step	Verified	<i>Inductive step: Verifies that all operations preserve the invariant that the validator count never exceeds the maximum validators limit.</i>	
------------------------------------	----------	---	--

P-S03. Validator Management Integrity

Status: Verified	Verifies integrity properties of validator registry operations. For all rules included in this property, we allow all loops to be unrolled at most twice.
------------------	---

Rule Name	Status	Description	Link to rule report
can_add_correct	Verified	<i>Verifies that validators can only be added to the registry through <code>get_or_add_validator_index_by_staking_pool_id_mut</code>, <code>join_stake</code>, and <code>join_fungible_stake</code>.</i>	Report
can_remove_correct	Verified	<i>Verifies that validators can only be removed from the registry through the refresh operation, which cleans up empty validators.</i>	
add_at_most_one	Verified	<i>Verifies that any single operation can add at most one validator to the registry.</i>	
no_inactive_stake_after_refresh	Verified	<i>Verifies that after a refresh operation completes, no validators in the registry contain inactive stake, ensuring all stake has been properly activated or removed.</i>	
no_empty_validators_after_refresh	Verified	<i>Verifies that after a refresh operation completes, no validators in the registry are empty (containing no stake), ensuring clean state and accurate registry size.</i>	

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.